04 · Context management & serialization

Siddharth M. Bhatia · siddharth@sidmb.com
July 1, 2026

Contents

Context-window policies 1
Serialization: recipe + transcript 2
Save / load a whole conversation (recipe + transcript) 3

Context-window policies

Long conversations overflow the model's context. A `ContextPolicy` decides what to keep. Crucially it runs on the **typed segments** (system / moderator / private_context / turns) *before* the family flatten, so framing is preserved reliably.

```
from interlens import (
    Conversation, ErrorPolicy, DropOldestPolicy, SlidingWindowPolicy,
     SummarizePolicy,
)

# ErrorPolicy (default): raise if the view exceeds context_limit — never
     silently truncate.
conv = Conversation(participants=(alice, bob),
     context_policy=ErrorPolicy(), context_limit=8192)

# SlidingWindowPolicy: keep framing + the most recent `keep_last` turns;
     drop older ones.
conv = Conversation(participants=(alice,
     bob), context_policy=SlidingWindowPolicy(keep_last=8))

# DropOldestPolicy: drop oldest turns until it fits (needs context_limit).
conv = Conversation(participants=(alice, bob),
     context_policy=DropOldestPolicy(), context_limit=8192)

# SummarizePolicy: replace older middle turns with a summary; keep framing
     + last `keep_last` verbatim.
conv = Conversation(
    participants=(alice, bob),
    context_policy=SummarizePolicy(keep_last=4, summarizer=lambda turns:
     "Earlier: " + " | ".join(turns)),
)
```

`context_limit=None` (default) means the tokenizer's own `model_max_length` is used.

Serialization: recipe + transcript

There is no separate template type: a `Conversation` **is** its own recipe. Because its participants are lazy (they hold an HF id + settings, not weights), an unrun conversation is a cheap, serializable spec — build it, ship it to workers, or run it many times, all as one object.

```
from interlens import Conversation, AutoModelParticipant,
     SlidingWindowPolicy
conv = Conversation(
    participants=[
        AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-3B-Instruct",
     name="alice", temperature=0.7, system_prompt="Be terse."),
        AutoModelParticipant.from_pretrained("google/gemma-2-2b-it",
     name="bob", temperature=0.9),
    ],
    shared_context="Debate: is a hotdog a sandwich?",
    shared_system_prompt="Stay civil.",
    context_policy=SlidingWindowPolicy(keep_last=8),
    reasoning_visibility="strip",
).turns(6)                                  # rollout/data fields via
     dot-modifier sugar
conv.run(turns=6)                           # loads the models lazily on
     first use
conv.save("runs/hotdog")                    # writes conversation.json
     (recipe) + transcript.json
later = Conversation.load("runs/hotdog")    # rebuilds lazy participants
     + attaches the transcript
```

`save` records each participant's own constructor kwargs (HF id + `dtype`/`attn`/`quant`/`revision`/`max_new_tokens`/`temperature`/`top_p`/`seed`/`thinking`/tool names/`max_tool_iters`/`kv_reuse`, or an API provider + model id) plus the scenario framing/policies — never weights. `load` raises on an unsupported (older) schema rather than silently mis-reading it.

Copy-on-write updates: `conv.set(field=value)` (or a dot-modifier like `conv.turns(6)`) returns a modified *copy* sharing the loaded models by reference — the original is untouched.

Save / load a whole conversation (recipe + transcript)

```
conv.save("runs/debate_001")                    # writes
    conversation.json (recipe) + transcript.json
resumed = Conversation.load("runs/debate_001", devices="cuda")   #
    rebuilds lazy participants, ATTACHES the transcript
resumed.run(turns=4)                            # continues from where it
    left off (does not regenerate)
```

`load` takes `devices=` as a single device or a list (participants are round-robined across the list — handy for putting two big models on two GPUs), and raises on an unsupported (older) schema rather than silently mis-reading it.

Next: tools · hooks · interpretability · rollouts.

© 2026 Siddharth M. Bhatia. All rights reserved. Interlens is licensed under the GNU Affero General Public License v3.0.